

Day 56: Sequential Data

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 9: Sequence Data & RNNs I (Day 1 of 7)

0. Where Today Fits

Week 9 begins a new theme: sequence data, where ORDER carries meaning. Every network this series has built so far -- dense (Days 36-41), CNN (Days 42-55) -- either flattens its input into a fixed-size vector or slides a fixed-size window over it. Today measures, rather than asserts, exactly where and why that breaks down on sequential data.

How to read each section

Every experiment below is laid out in the same order: **Specification** (exactly what was set up -- data, sizes, settings, what counts as success), **Why this test** (what question it answers), **Implementation** (the real code from the lesson script that produces the result), **Actual output** (copied verbatim from the run output file), and **What it means**. You should be able to predict each output from the first three before you read it.

Watch: [Recurrent Neural Networks \(RNNs\), Clearly Explained!!!](#)

Watch: [CNN Receptive Field | Deep Learning Animated](#)

1. The Task -- Order Matters, Values Don't

SPECIFICATION	
Input	a sequence of T scalars: Gaussian noise (std 0.3) plus two spikes
Spikes	one HIGH spike (+3) and one LOW spike (-3) at two DISTINCT random positions in [pos_lo, pos_hi]
Label	1 if the high spike comes BEFORE the low spike in time, 0 if after
Balance	n_per_class examples of each label, then shuffled together
Key property	the same two values are present either way -- ONLY their relative order decides the label

Why this task

It isolates the defining property of sequential data. A network cannot solve it by noticing WHICH values appear -- both classes contain exactly one +3 and one -3. It must represent which came first, like "price went up then down" versus "down then up". The pos_lo/pos_hi arguments let later sections control exactly WHERE spikes are allowed to appear.

IMPLEMENTATION

```
def make_order_dataset(n_per_class, T, pos_lo, pos_hi, seed):
    rng = np.random.RandomState(seed)
    X, y = [], []
    for label in [0, 1]:
        for _ in range(n_per_class):
            seq = rng.randn(T) * 0.3
            p1, p2 = rng.choice(np.arange(pos_lo, pos_hi + 1), size=2, replace=False)
            p_high, p_low = (p1, p2) if p1 < p2 else (p2, p1) # high = EARLIER
            if label == 0:
                p_high, p_low = p_low, p_high # swap: low comes first
            seq[p_high] += 3.0
            seq[p_low] -= 3.0
            X.append(seq); y.append(label)
    X, y = np.array(X), np.array(y)
    idx = rng.permutation(len(y))
    return X[idx], y[idx]
```

Sorting p1, p2 first and then swapping for label 0 guarantees, by construction, that the label always matches the spikes' order -- no other cue exists for a network to exploit as a shortcut.

2. PlainFCNet and the Fixed-Length Limitation

SPECIFICATION

Network	PlainFCNet: input T -> dense hidden layer (16, ReLU) -> dense output (2) -> softmax
Key parameter	W1 of shape (T, hidden) -- ONE ROW PER INPUT POSITION
Built for	T = 30 (PlainFCNet(T=30, seed=42))
Fed	a batch of 4 sequences of length 20: randn(4, 20), seed 0
Training	none -- this is an architectural check
Expected	a ValueError, because 20 != 30

Why this test

Real sequences come in different lengths -- sentences, recordings, sensor logs. A dense network's input size is baked into W1's shape at construction time, so it cannot even accept a different length, however well it was trained. PlainFCNet checks the length explicitly so the failure is a clear message rather than a confusing numpy shape error.

IMPLEMENTATION

```
class PlainFCNet:
    def __init__(self, T, hidden=16, n_classes=2, lr=0.2, seed=42):
        rng = np.random.RandomState(seed)
        self.T = T                    # fixed at construction
        self.W1 = rng.randn(T, hidden) * he_scale(T)    # one row per position
        ...
    def forward(self, X):
        if X.shape[1] != self.T:
            raise ValueError(f"PlainFCNet was built for sequence length {self.T}, "
                             f"but received length {X.shape[1]} -- ...")

net = PlainFCNet(T=30, seed=42)
short_seq = np.random.RandomState(0).randn(4, 20)
try:
    net.forward(short_seq)
except ValueError as e:
    print(f"ValueError raised, as expected: {e}")
```

ACTUAL OUTPUT

```
ValueError raised, as expected: PlainFCNet was built for sequence length 30, but
received length 20 -- a dense network's input size is fixed at construction time.
```

What it means

No training can fix this -- it is a property of the architecture's shape. An RNN (Day 57) has no T in its constructor at all, because it reuses the same small set of weights once per timestep.

3. Failure Mode #1 -- PlainFCNet Can't Generalize Across Positions

SPECIFICATION

Sequence length	T = 30
Training data	200 sequences (100 per class, seed 42), spikes ONLY in positions 0-14
Seen-position test	80 sequences (40 per class, seed 123), spikes in positions 0-14 -- held out
Shifted test	80 sequences (40 per class, seed 999), spikes ONLY in positions 15-29 -- never seen
Network	PlainFCNet(T=30, hidden=16, lr=0.2, seed=7)
Training	200 epochs, mini-batch size 40, reshuffled each epoch (seed 7), plain SGD
Chance level	accuracy 0.5

Why this test

The rule is identical everywhere in the sequence -- only its location changes. The seen-position test is the fair baseline (did it learn the rule at all?); the shifted test asks whether that learning transfers to a new location. The implementation note shows why it shouldn't.

IMPLEMENTATION

```
# inside PlainFCNet.backward():
dw1 = self.X.T @ dZ1 / m          # row t of dw1 is nonzero ONLY where input position t
                                   # carried signal in this batch

# training loop
net = PlainFCNet(T=T, hidden=16, lr=0.2, seed=7)
epochs, batch_size = 200, 40
rng = np.random.RandomState(7)
for epoch in range(epochs):
    order = rng.permutation(len(y_train))
    for start in range(0, len(order), batch_size):
        idx = order[start:start + batch_size]
        net.forward(X_train[idx])
        net.backward(one_hot(y_train[idx], 2))

tr_acc      = net.accuracy(X_train, y_train)
seen_acc    = net.accuracy(X_test_seen, y_test_seen)
shifted_acc = net.accuracy(X_test_shifted, y_test_shifted)
```

During training, positions 15-29 only ever hold small noise, so rows 15-29 of W1 receive almost no useful gradient and stay close to their random starting values.

ACTUAL OUTPUT

```
train_acc (seen positions):      1.0000
test_acc (seen positions, held out):  0.8875
test_acc (SHIFTED positions 15-29):  0.5500
```

What it means

PlainFCNet genuinely learned the rule (1.000 train, 0.888 on held-out seen positions) -- then collapsed to 0.550, chance level, when the identical pattern moved. Nothing about the task changed except WHERE. A dense network has no mechanism for a pattern learned at one position to transfer to another.

4. Conv1DNet -- and Checking Its One New Piece of Math

SPECIFICATION

Architecture	reshape (m, T) -> (m, 1, 1, T); conv with filters Wc of shape (C, 1, 1, k), no padding -> (m, C, L) with L = T - k + 1; ReLU; global average pool over L -> (m, C); linear C -> 2; softmax
Reused code	conv2d_forward_mc / conv2d_backward_mc from Day 53, unchanged -- already gradient-checked there
New code	global average pooling, forward and backward
Gradient-check network	Conv1DNet(T=12, C=3, k=3, n_classes=2, lr=0.01, seed=1)
Check input	4 random sequences: randn(4, 12) * 0.5 (seed 1); labels [0, 1, 0, 1]
Checked	Wc and W1, 3 random entries each; central difference, eps = 1e-4
Pass criterion	max relative error < 1e-4

Why this test

A CNN shares ONE set of filter weights across every position -- exactly the translation invariance PlainFCNet lacked. The reshape trick turns a 1D sequence into a 1-pixel-tall image so Day 53's verified 2D convolution does the 1D work with no new conv math. The only genuinely new piece is global average pooling's backward pass, so that is what gets verified before any result built on it is trusted.

IMPLEMENTATION

```
def global_avg_pool_forward(A):          # (m, C, L) -> (m, C)
    return A.mean(axis=2)

def global_avg_pool_backward(dOut, L):    # each of the L positions gets an equal 1/L share
    return np.repeat(dOut[:, :, None], L, axis=2) / L

# Conv1DNet.forward
self.Xr = X.reshape(m, 1, 1, self.T)    # sequence -> 1-tall image
Zc = conv2d_forward_mc(self.Xr, self.Wc, self.bc, pad=0) # (m, C, 1, L)
self.Zc = Zc.reshape(m, Zc.shape[1], Zc.shape[3])      # squeeze -> (m, C, L)
self.Ac = relu(self.Zc)
self.pooled = global_avg_pool_forward(self.Ac)          # (m, C) -- position is gone
self.probs = softmax(self.pooled @ self.W1 + self.b1)

# Conv1DNet.backward (the part that differs from Day 53)
dpooled = dZ1 @ self.W1.T
dAc = global_avg_pool_backward(dpooled, L)
dZc = dAc * drelu(self.Zc)
dWc, dbc, _ = conv2d_backward_mc(dZc.reshape(m, dZc.shape[1], 1, L), self.Xr, self.Wc)
```

The mean's derivative: every one of the L positions contributes with weight 1/L, so the gradient flowing back is split equally -- dOut / L to each position.

ACTUAL OUTPUT

param	analytic	numeric	rel_err
Wc(0, 0, 0, 1)	-0.087497	-0.087497	5.03e-09
Wc(1, 0, 0, 0)	-0.117464	-0.117464	5.08e-09
Wc(0, 0, 0, 0)	0.051447	0.051447	8.40e-09
W1(2, 0)	-0.108058	-0.108058	6.92e-09
W1(1, 0)	-0.268046	-0.268046	5.74e-09
W1(1, 0)	-0.268046	-0.268046	5.74e-09

max relative error across all checked entries: 8.40e-09 (PASS -- Conv1DNet backward chain (incl. global average pooling) is correct)

What it means

Agreement to about eight significant figures on both parameters: the pooling backward pass is correct, so Section 5's result reflects the architecture, not a bug.

5. Failure Mode #2 -- Conv1DNet's Capped Receptive Field

SPECIFICATION

Sequence length / kernel	T = 30, k = 5 -- one filter position sees 5 consecutive inputs (its receptive field)
Data	train 300 (150 per class, seed 42), test 300 (seed 999); spikes anywhere in 0-29 for BOTH -- no shift
Network	Conv1DNet(T=30, C=16, k=5, lr=1.0, seed=7)
Training	800 epochs, mini-batch size 50, plain SGD
Split	after training, each test example is 'near' if $ p_{\text{high}} - p_{\text{low}} \leq 5$ (both spikes fit in one filter window) or 'far' if > 5 (they never do)
Chance level	accuracy 0.5

Why this test

Weight sharing fixes Section 3's problem, but each filter still only sees k positions at once. For a near pair, some window contains both spikes and a filter can detect their order directly. For a far pair, every window sees at most one spike -- and global average pooling then discards WHERE each detection happened. Splitting one test set by distance measures that ceiling directly.

IMPLEMENTATION

```
net = Conv1DNet(T=T, C=16, k=k, lr=1.0, seed=7)
# ... 800 epochs of mini-batch SGD, as in Section 3 ...

preds = net.forward(X_test).argmax(axis=1)
distances = []
for seq in X_test:
    p_high = int(np.argmax(seq))      # the +3 spike is the sequence maximum
    p_low = int(np.argmin(seq))       # the -3 spike is the minimum
    distances.append(abs(p_high - p_low))
distances = np.array(distances)
near_mask = distances <= k
far_mask = distances > k
near_acc = (preds[near_mask] == y_test[near_mask]).mean()
far_acc = (preds[far_mask] == y_test[far_mask]).mean()
```

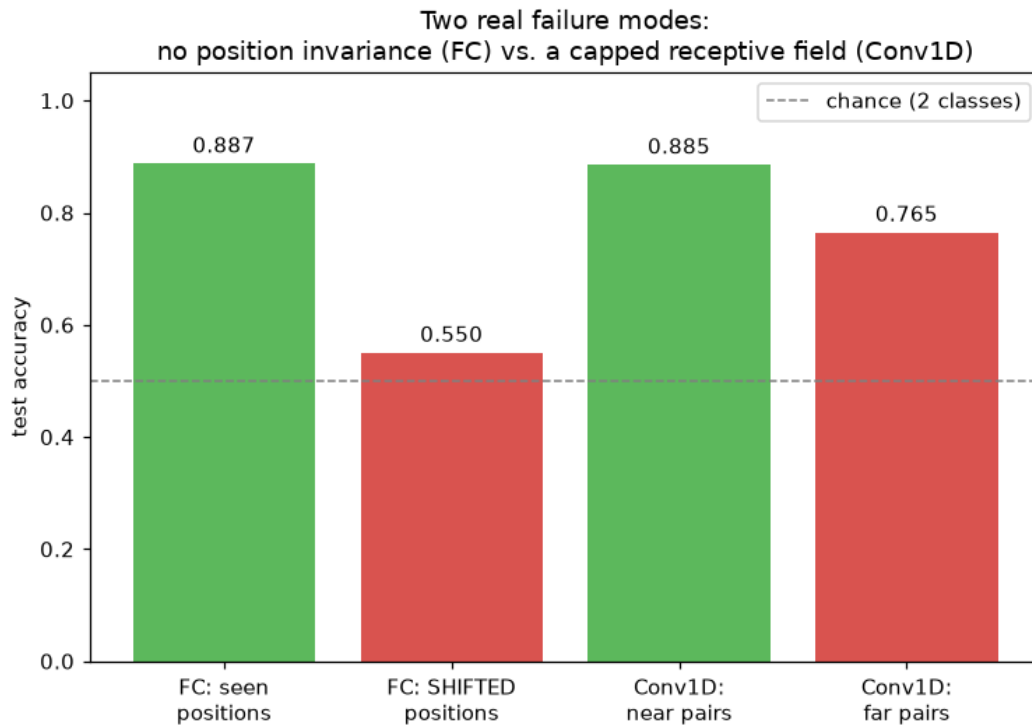
Spike positions are recovered from the test sequences themselves with argmax/argmin -- safe because the +-3 spikes dwarf the 0.3-std noise.

ACTUAL OUTPUT

overall train_acc: 0.9100 overall test_acc: 0.8033

ACTUAL OUTPUT

test_acc on 'near' pairs (spike distance <= 5, n=96): 0.8854
test_acc on 'far' pairs (spike distance > 5, n=204): 0.7647



What it means

Near pairs: 0.885. Far pairs: 0.765 -- a real, repeatable gap of 0.121. But not a collapse to chance. Pooling discards relative position, yet a spike near either end of the sequence falls in fewer sliding windows than one in the middle, so the pooled average still leaks a faint absolute-position signal the network partially exploits. The ceiling is real, just 'degraded' rather than 'destroyed' -- softer than PlainFCNet's collapse.

WHY THIS CHOICE

A correction made along the way: the first configuration (lr=0.2, 200 epochs, C=8) left even TRAINING accuracy under 80%, and produced the OPPOSITE pattern -- far pairs scoring higher than near. An undertrained network can't support a near/far comparison. Retraining harder (lr=1.0, 800 epochs, C=16), checked directly, reached 0.910 training accuracy and the expected pattern. Always confirm the model actually trained before interpreting a breakdown of its results.

6. What's Needed Instead

PlainFCNet keeps position-specific weights (no invariance); Conv1DNet shares weights but only within a local window, then discards position. The task needs a network that walks through the sequence ONE STEP AT A TIME, reuses the SAME weights at every step, and carries a running summary -- a hidden state -- forward, so position 2 can still influence the decision at position 29. That is a Recurrent Neural Network, built in Day 57.

Interview Preparation -- Technical Questions

Q: Why does a dense network fail to generalize a pattern across a position shift?

A: Its first-layer weights have one row per input position, and a row only learns from gradient signal that position actually carries. Rows for positions that never carried signal in training stay near random -- measured here as a drop from 0.888 (seen positions) to 0.550 (shifted).

Q: Why does a 1D CNN do better across shifts, and what limitation remains?

A: Its filter weights are shared across every position, giving translation invariance. But each filter application sees only k positions; events farther apart than k are never jointly seen -- measured as 0.885 (near) vs. 0.765 (far).

Q: Why weren't the far pairs at chance level?

A: Global average pooling removes relative order but not all positional information: spikes near the ends appear in fewer windows than spikes in the middle, so the pooled average carries a faint absolute-position signal the network can partially use.

Q: What is the backward pass of global average pooling?

A: The mean weights each of L positions by $1/L$, so the incoming gradient is copied to every position and divided by L : `np.repeat(dOut[:, :, None], L, axis=2) / L`.

Interview Preparation -- Theory Questions

Q: What makes data 'sequential'?

A: Order carries information beyond which values are present -- today's two classes contain identical values and differ only in order.

Q: Which two properties must a sequence model have, based on today's failures?

A: Translation invariance across time (the same weights at every position -- fixes PlainFCNet) and no hard limit on how far information can travel (fixes Conv1DNet's receptive field).

Q: Why measure both failures instead of saying 'dense and CNN nets fail on sequences'?

A: Because they fail differently -- one collapses to chance, the other degrades -- and that difference matters when choosing between a deeper CNN and an RNN for a real problem.

Key Takeaway and What's Next

Key takeaway: sequence data breaks fixed-vector and fixed-window architectures in two measured ways -- no position invariance (a collapse to chance on shifted patterns) and a capped receptive field (a real but softer degradation on far-apart events). Both point to one missing ingredient: identical processing at every step, with information free to travel the whole sequence.

What's next: Day 57 builds a vanilla RNN's forward pass from scratch.

Suggested watch order: 1) Recurrent Neural Networks, Clearly Explained!!!, 2) CNN Receptive Field.